

ORDER MANAGEMENT

1. Question

Java Program Documentation

Write a Java program to demonstrate inheritance using Order, SpecialOrder and NormalOrder classes, and customer interaction with orders.

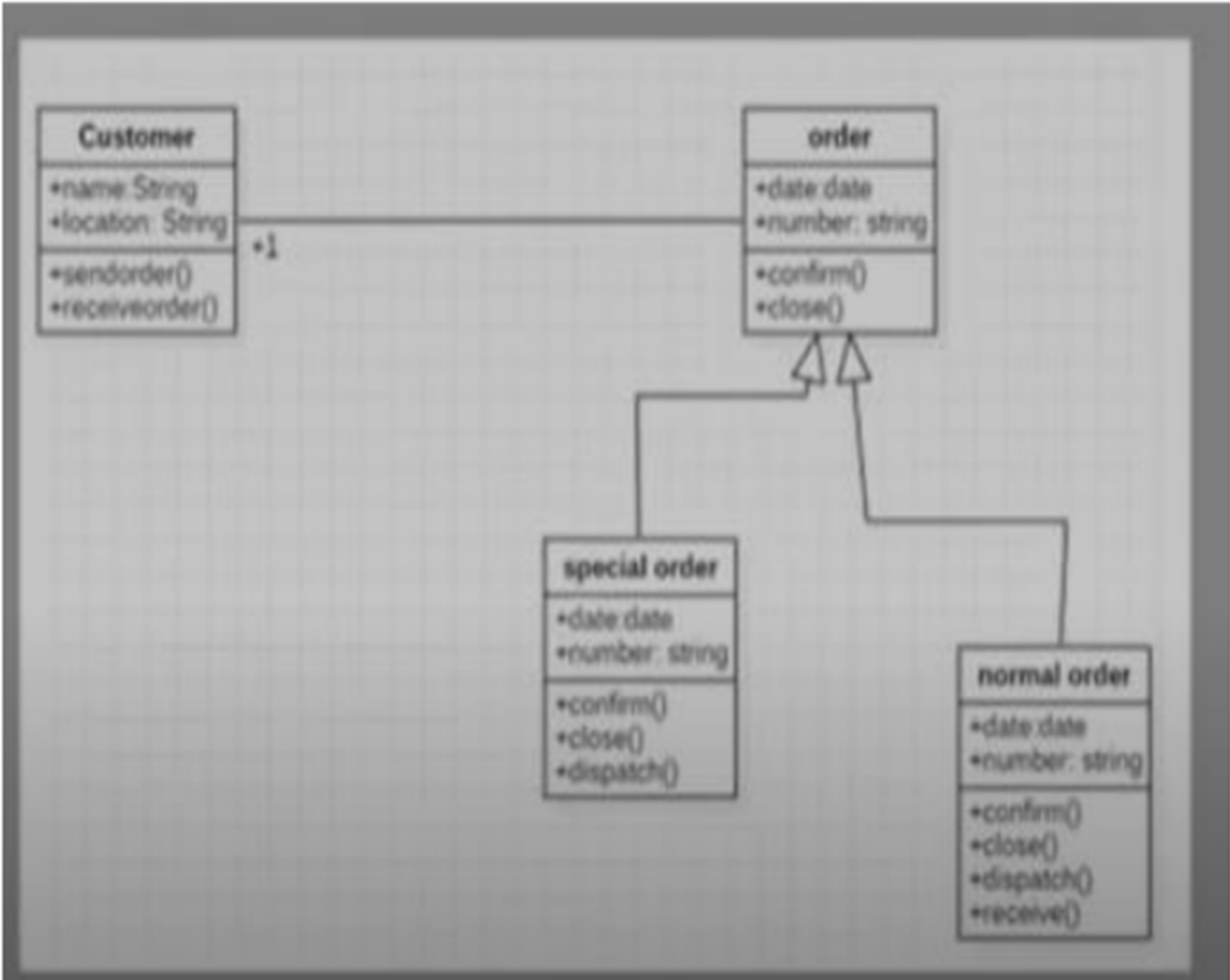
2. Execution Steps

1. Open Eclipse IDE or a terminal.
2. Create a Java project/package named Excercise.
3. Create the Java classes Order, SpecialOrder, NormalOrder, customer and MainOrder.
4. Copy the complete program into MainOrder.java.
5. Make sure the public class name is MainOrder.
6. Save the program.
7. Compile the program.
8. Run the MainOrder class.
9. Observe confirmation, dispatch, receiving and closing messages for both orders.
10. Verify the output in the terminal.

3. Algorithm

1. Start.
2. Define the Order class with date and number, constructor, confirm(), close(), getters and setters.
3. Define SpecialOrder as a subclass of Order and override/implement dispatch().
4. Define NormalOrder as a subclass of Order and implement dispatch() and receive().
5. Define the customer class with name and location, constructor, sendOrder(), receiveOrder(), getters and setters.
6. In main(), create a customer named joe from anits.
7. Create a SpecialOrder with date 13 august and number sp-1.
8. Send, confirm, dispatch, receive and close the special order.
9. Create a NormalOrder with date 16 august and number no-1.
10. Confirm, dispatch, receive, receive and close the normal order.
11. Receive the special order once more.
12. Stop.

4. UML Class Diagram



5. Java Source Code

```
package Excercise;

class Order{
    public String date;
    public String number;
    public Order(String date,String number) {
        this.date=date;
        this.number=number;
    }
    public void confirm() {
        System.out.println("order"+number+"confirmed");
    }
    public void close() {
        System.out.println("order"+number+"closed");
    }
    public String getDate () { return date; }
    public void setDate (String date) { this.date=date; }
    public String getNumber() { return number; }
    public void setNumber(String number) { this.number=number; }
}

class SpecialOrder extends Order{
    SpecialOrder(String date,String number){ super(date,number); }
    public void dispatch() {
        System.out.println("Special order " + number + " dispatched");
    }
}

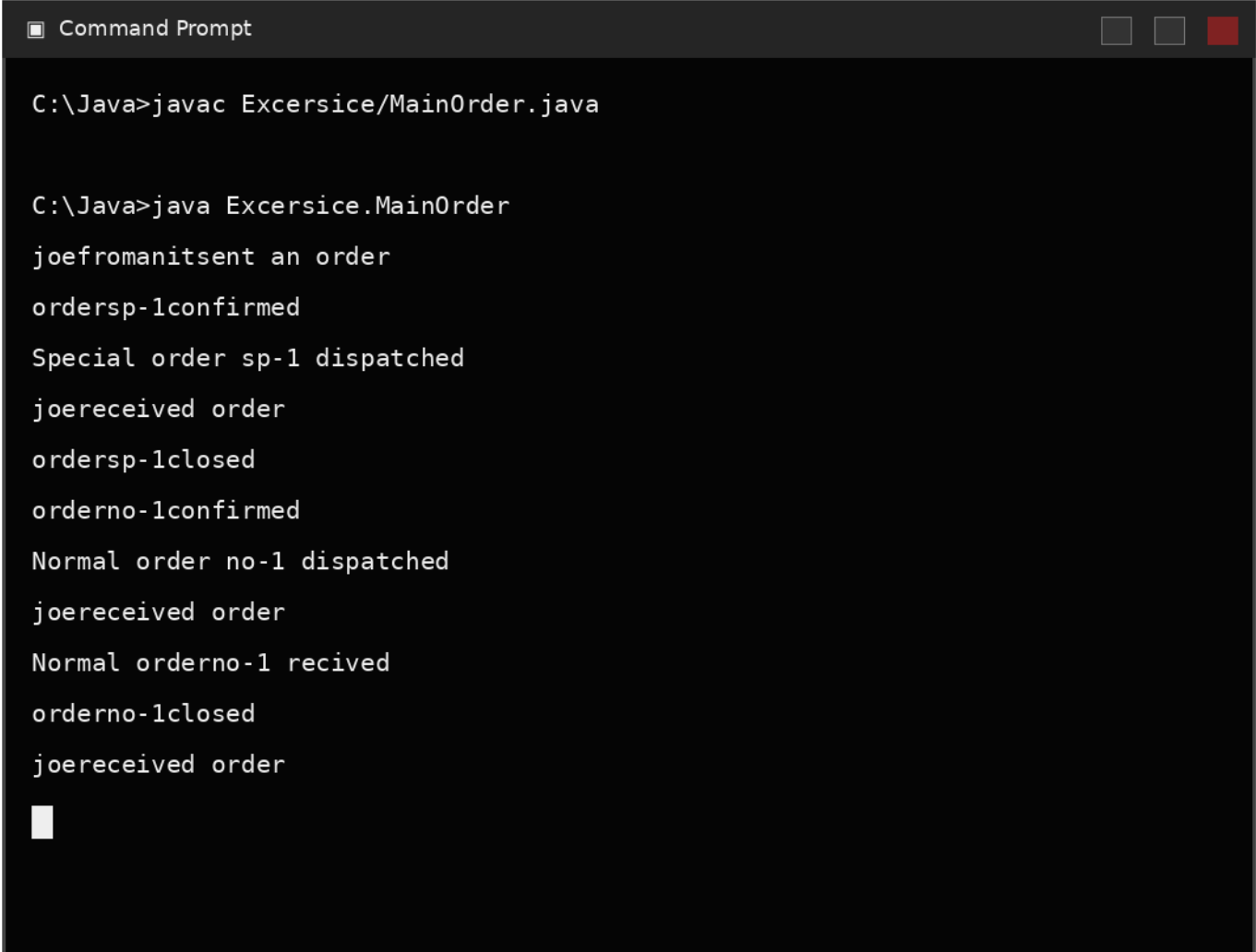
class NormalOrder extends Order{
    NormalOrder(String date,String number){ super(date,number); }
    public void dispatch() {
        System.out.println("Normal order " + number + " dispatched");
    }
    public void receive() {
        System.out.println("Normal order"+ number + " recived");
    }
}
```

5. Java Source Code

```
class customer{
    private String name;
    private String location;
    customer(String name, String location) {
        this.name = name;
        this.location = location;
    }
    public void sendOrder(Order order) {
        System.out.println(name+" "+"from"+" "+location+" "+"sent an order");
    }
    public void receiveOrder(Order order) {
        System.out.println(name+" "+"received order");
    }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getLocation() { return location; }
    public void setLocation(String location) { this.location = location; }
}

public class MainOrder {
    public static void main(String[] args) {
        customer c = new customer("joe","anits");
        SpecialOrder s = new SpecialOrder("13 august","sp-1");
        c.sendOrder(s);
        s.confirm();
        s.dispatch();
        c.receiveOrder(s);
        s.close();
        NormalOrder n = new NormalOrder("16 august","no-1");
        n.confirm();
        n.dispatch();
        c.receiveOrder(n);
        n.receive();
        n.close();
        c.receiveOrder(s);
    }
}
```

6. Output – Terminal Screenshot



```
Command Prompt

C:\Java>javac Excercise/MainOrder.java

C:\Java>java Excercise.MainOrder
joefromanitsent an order
ordersp-1confirmed
Special order sp-1 dispatched
joereceived order
ordersp-1closed
orderno-1confirmed
Normal order no-1 dispatched
joereceived order
Normal orderno-1 recived
orderno-1closed
joereceived order
█
```

The screenshot shows a Windows Command Prompt window titled "Command Prompt". The user has navigated to the directory C:\Java. They first compile the Java file Excercise/MainOrder.java using the javac command. Then, they run the compiled class using the java command. The output of the program is displayed line by line, showing a sequence of events related to order processing: "joefromanitsent an order", "ordersp-1confirmed", "Special order sp-1 dispatched", "joereceived order", "ordersp-1closed", "orderno-1confirmed", "Normal order no-1 dispatched", "joereceived order", "Normal orderno-1 recived", "orderno-1closed", and "joereceived order". The prompt ends with a white cursor block.